

Magíster en Ciencia de Datos
para la Innovación

FUNDAMENTOS DE BASES DE DATOS Y ALGORITMOS: Parte 1

JOSÉ FUENTES

Ingeniero Civil Informático

Doctor en Ciencias de la Computación

jfuentess@inf.udec.cl

CLASE #2

Tipos de datos compuestos



Facultad de Ingeniería
Universidad de Concepción



CENTRO DE DATOS E
INTELIGENCIA ARTIFICIAL
UNIVERSIDAD DE CONCEPCIÓN

Tipos de datos compuestos

Estructuras de datos nativas

Listas (list)

0	1	2	3	4	5
18.8	1.7	16.9	5.9	214.3	336.2

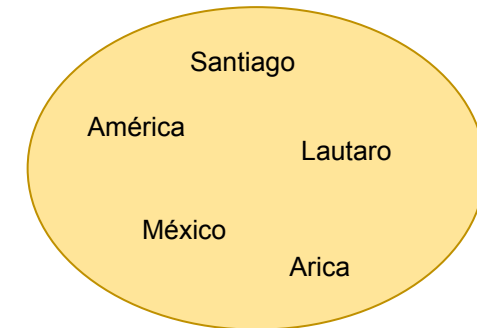
Diccionarios (dict)

Clave	Valor
Dólar Observado	697.19
Dólar Canadiense	552.40
Euro	838.47
Peso Argentino	7.50

Tuplas (tuple)

	latitud	longitud
Valparaíso	-33.036	-71.62963
Temuco	-38.73965	-72.59842
Concepción	-36.82699	-73.04977

Conjuntos (set)



Listas

Total precipitación mensual en mm, año 2019. Estación Cerro Caracol, Concepción
Fuente: DGAC

	0	1	2	3	4	5	6	7	8	9	10	11
L =	18.8	1.7	16.9	5.9	214.3	336.2	144.4	68.6	79.1	42.7	18.9	0.0

Notación Python

```
L = [18.8, 1.7, 16.9, 5.9, 214.3, 336.2, 144.4, 68.6, 79.1, 42.7, 18.9, 0.0]
```

	0	1	2	3	4	5	6
M =	Edmundo	Larenas	219	-36.83058	-73.03687	True	None

Notación Python

```
M = ['Edmundo', 'Larenas', 219, -36.83058, -73.03687, True, None]
```

Listas: agregar valores

	0	1	2	3	4	5	6	7	8	9	10	11
L =	18.8	1.7	16.9	5.9	214.3	336.2	144.4	68.6	79.1	42.7	18.9	0.0

- Agregar la precipitación de enero 2020: 19.5 mm

```
L.append(19.5)
```

	0	1	2	3	4	5	6	7	8	9	10	11	12
L =	18.8	1.7	16.9	5.9	214.3	336.2	144.4	68.6	79.1	42.7	18.9	0.0	19.5

- Agregar las precipitaciones de enero, febrero y marzo 2020: 19.5 mm, 0.8 mm y 15.1 mm

```
L.extend([19.5, 0.8, 15.1])
```

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
L =	18.8	1.7	16.9	5.9	214.3	336.2	144.4	68.6	79.1	42.7	18.9	0.0	19.5	0.8	15.1

- Agregar el valor 100.5 en la posición 8

```
L.insert(8, 100.5)
```

	0	1	2	3	4	5	6	7	8	9	10	11	12
L =	18.8	1.7	16.9	5.9	214.3	336.2	144.4	68.6	100.5	79.1	42.7	18.9	0.0

Listas: eliminar valores

	0	1	2	3	4	5	6	7	8	9	10	11
L =	18.8	1.7	16.9	5.9	214.3	336.2	144.4	68.6	79.1	42.7	18.9	0.0

- Eliminar y retorna el último valor

```
L.pop()
```

	0	1	2	3	4	5	6	7	8	9	10	retorno
L =	18.8	1.7	16.9	5.9	214.3	336.2	144.4	68.6	79.1	42.7	18.9	0.0

- Eliminar y retorna el valor en la posición 4

```
L.pop(4)
```

	0	1	2	3	4	5	6	7	8	9	10	retorno
L =	18.8	1.7	16.9	5.9	336.2	144.4	68.6	79.1	42.7	18.9	0.0	214.3

Listas: eliminar valores

	0	1	2	3	4	5	6	7	8	9	10	11
L =	18.8	1.7	16.9	5.9	214.3	336.2	144.4	68.6	79.1	42.7	18.9	0.0

- Elimina el valor 214.3 (en caso de existir más de una entrada con ese valor, elige la que tiene índice menor)

```
L.remove(214.3)
```

	0	1	2	3	4	5	6	7	8	9	10
L =	18.8	1.7	16.9	5.9	336.2	144.4	68.6	79.1	42.7	18.9	0.0

- Elimina el valor en la posición 4

```
del L[4]
```

	0	1	2	3	4	5	6	7	8	9	10
L =	18.8	1.7	16.9	5.9	336.2	144.4	68.6	79.1	42.7	18.9	0.0

- Elimina todos los valores

```
L.clear()
```

L = []

Listas: otros métodos útiles

	0	1	2	3	4	5	6	7	8	9	10	11
L =	18.8	1.7	16.9	5.9	214.3	336.2	144.4	68.6	79.1	42.7	18.9	0.0

- Contar cuántas veces aparece el valor 1.7

```
L.count(1.7)
```

Retorno: 1

- Retorna la posición donde está el valor 144.4 (en caso de no existir, genera un error)

```
L.index(144.4)
```

Retorno: 6

- Retorna la cantidad de elementos

```
len(L)
```

Retorno: 12

- Reemplazar/actualizar el valor de la posición 3 por 6.1

```
L[3] = 6.1
```

	0	1	2	3	4	5	6	7	8	9	10	11
L =	18.8	1.7	16.9	6.1	214.3	336.2	144.4	68.6	79.1	42.7	18.9	0.0

- Comprueba si el valor 79.1 está en la lista

```
79.1 in L
```

Retorno: True

- Ordenar los valores de la lista

```
L.sort()
```

	0	1	2	3	4	5	6	7	8	9	10	11
L =	0.0	1.7	6.1	16.9	18.8	18.9	42.7	68.6	79.1	144.4	214.3	336.2

Diccionario

Valores al 21 de abril, 2019

Clave	Valor
Dólar Observado	697.19
Dólar Canadiense	552.40
Euro	838.47
Peso Argentino	7.50
Sol Peruano	188.96

Notación Python

```
tipo_cambio = {  
    'dolar_observado': 697.19,  
    'dolar_canadiense': 552.4,  
        'euro': 838.47,  
    'peso_argentino': 7.5,  
    'sol_peruano': 188.96}
```

Valores al 19, 20 y 21 de abril, 2019

Clave	Valor		
Dólar Observado	701.55	700.96	697.19
Dólar Canadiense	561.06	559.11	552.40
Euro	840.08	843.62	838.47
Peso Argentino	7.56	7.54	7.50
Sol Peruano	193.12	190.91	188.96

Notación Python

```
tipo_cambio = {  
    'dolar_observado': [701.55, 700.96, 697.19],  
    'dolar_canadiense': [561.06, 559.11, 552.4],  
        'euro': [840.08, 843.62, 838.47],  
    'peso_argentino': [7.56, 7.54, 7.5],  
    'sol_peruano': [193.12, 190.91, 188.96]}
```

* Valores obtenidos del Banco central (www.bcentral.cl)

Diccionario

tipo_cambio =

Clave	Dólar Observado	Dólar Canadiense	Euro	Peso Argentino	Sol Peruano
Valor	697.19	552.40	838.47	7.50	188.96

- Agregar 'Real Brasileño' con valor 125.89

```
tipo_cambio['real_brasileno'] = 125.89
```

Clave	Dólar Observado	Dólar Canadiense	Euro	Peso Argentino	Sol Peruano	Real Brasileño
Valor	697.19	552.40	838.47	7.50	188.96	125.89

- Actualizar el valor de 'Euro' a 840.59

```
tipo_cambio['euro'] = 840.59
```

Clave	Dólar Observado	Dólar Canadiense	Euro	Peso Argentino	Sol Peruano
Valor	697.19	552.40	840.59	7.50	188.96

- Elimina la entrada para 'Euro'

```
del tipo_cambio['euro']
```

Clave	Dólar Observado	Dólar Canadiense	Peso Argentino	Sol Peruano
Valor	697.19	552.40	7.50	188.96

Diccionario

tipo_cambio =

Clave	Dólar Observado	Dólar Canadiense	Euro	Peso Argentino	Sol Peruano
Valor	697.19	552.40	838.47	7.50	188.96

- Recuperar una lista con las claves

```
L = tipo_cambio.keys()
```

```
L = ['dolar_observado', 'dolar_canadiense', 'euro', 'peso_argentino', 'sol_peruano']
```

- Recuperar una lista con los valores

```
L = tipo_cambio.values()
```

```
L = [697.19, 552.40, 838.47, 7.50, 188.96]
```

- Recuperar una lista con tuplas (clave, valor)

```
L = tipo_cambio.items()
```

```
L = [('dolar_observado', 697.19), ('dolar_canadiense', 552.40), ('euro', 838.47),  
      ('peso_argentino', 7.50), ('sol_peruano', 188.96)]
```

Diccionario

tipo_cambio =

Clave	Dólar Observado	Dólar Canadiense	Euro	Peso Argentino	Sol Peruano
Valor	697.19	552.40	838.47	7.50	188.96

- Retorna la cantidad de entradas

```
len(tipo_cambio)
```

Retorno: 5

- Comprueba que la clave 'Euro' exista en el diccionario

```
'euro' in tipo_cambio
```

Retorno: True

- Elimina todas las entradas del diccionario

```
tipo_cambio.clear()
```

```
tipo_cambio = {}
```

Tuplas (listas inmutables)

Coordenadas ciudades chilenas

	latitud	longitud
Santiago	-33.45694	-70.64827
Valparaíso	-33.036	-71.62963
Temuco	-38.73965	-72.59842
Concepción	-36.82699	-73.04977
Arica	-18.4746	-70.29792

Notación Python

```
santiago = (-33.45694, -70.64827)
valparaiso = (-33.036, -71.62963)
temuco = (-38.73965, -72.59842)
concepcion = (-36.82699, -73.04977)
arica = (-18.4746, -70.29792)
```

- Contar cuántas veces aparece la coordenada -33.036 en la tupla `valparaiso`

```
valparaiso.count(-33.036)
```

Retorno: 1

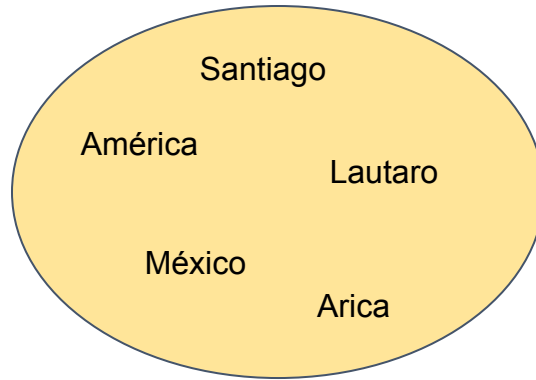
- Retorna la posición donde está la coordenada -18.4746 en la tupla `arica`

```
arica.index(-18.4746)
```

Retorno: 0

Conjuntos

Nombres de lugares geográficos

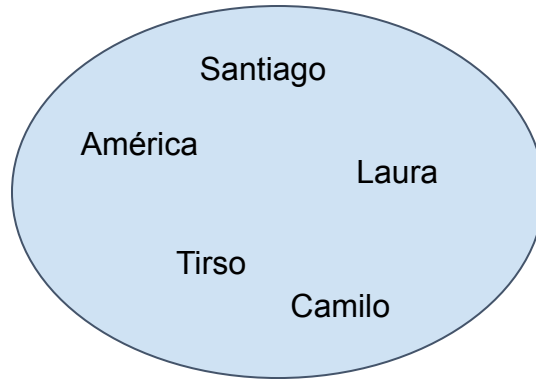


Notación Python

```
lugares = set(['Santiago', 'America', 'Lautaro', 'Mexico', 'Arica'])
```

```
lugares = {'Santiago', 'America', 'Lautaro', 'Mexico', 'Arica'}
```

Nombres de personas



Notación Python

```
nombres = set(['Santiago', 'America', 'Laura', 'Tirso', 'Camilo'])
```

```
nombres = {'Santiago', 'America', 'Laura', 'Tirso', 'Camilo'}
```

Conjuntos

Notación Python

```
lugares = set(['Santiago', 'America', 'Lautaro', 'Mexico', 'Arica'])
```

```
nombres = set(['Santiago', 'America', 'Laura', 'Tirso', 'Camilo'])
```

- Agregar 'Talca' al conjunto de lugares

```
lugares.add('Talca')
```

```
{'Santiago', 'America', 'Lautaro', 'Mexico', 'Arica', 'Talca'}
```

- Eliminar 'America' del conjunto de lugares

```
lugares.remove('America')
```

```
{'Santiago', 'Lautaro', 'Mexico', 'Arica'}
```

- Eliminar todas las entradas del conjunto lugares

```
lugares.clear()
```

```
{}
```

Conjuntos

Notación Python

```
lugares = set(['Santiago', 'America', 'Lautaro', 'Mexico', 'Arica'])
```

```
nombres = set(['Santiago', 'America', 'Laura', 'Tirso', 'Camilo'])
```

- Unión de conjuntos

```
lugares.union(nombres)
```

```
{'Laura', 'Tirso', 'America', 'Mexico', 'Lautaro', 'Santiago', 'Arica', 'Camilo'}
```

- Intersección de conjuntos

```
lugares.intersection(nombres)
```

```
{'America', 'Santiago'}
```

- Diferencia de conjuntos

```
lugares.difference(nombres)
```

```
{'Mexico', 'Lautaro', 'Arica'}
```

- Verifica si 'Rancagua' pertenece al conjunto de lugares

```
'Rancagua' in lugares
```

Retorno: False

¡A PRACTICAR!

